Clustering-Informed Retrieval-Augmented Generation for LLM-Based Log Anomaly Detection

Jielun Zhang*, Fuhao Li§, Hongyu Wu†, and Venkataramani Kumar‡
*School of Electrical Engineering & Computer Science, University of North Dakota, Grand Forks, ND, 58202, USA
§Computer Science Department, La Sierra University, Riverside, CA, 92505, USA
†Clarivate, Ann Arbor, MI, 48108, USA
‡Department of Computer Science and Engineering, University of Puerto Rico - Mayagüez, Puerto Rico, 00680, USA
Emails: *jielun.zhang@UND.edu, §fli@lasierra.edu, †hongyu.wu@clarivate.com, ‡venkataramani.kumar@upr.edu

***Abstract*—Retrieval-Augmented Generation (RAG) empowers large language models (LLMs) to perform context-aware reasoning by retrieving relevant examples at inference time. In log anomaly detection, the quality of retrieved examples is crucial to classification accuracy, especially when the retrieval pool is constrained either due to computational limits or storage considerations. In this work, we study how different retrieval sampling strategies affect LLM performance and propose clustering-informed methods based on K-Means. Specifically, we introduce centroid-based, edge-based, and hybrid strategies that aim to enhance the semantic diversity and coverage of retrieved contexts. Extensive evaluation has been conducted on OpenStack logs using *Granite3.2:8B* and *Mistral-Nemo:12B* hosted locally. The evaluation results demonstrate our methods consistently outperform baseline methods across F1 and accuracy. Our observation also highlights the importance of retrieval structure in RAG pipelines and demonstrates the value of clustering-aware example selection for LLM-based log anomaly detection.**

***Index Terms*—log anomaly detection, Large Language Model (LLM), Retrieval-Augmented Generation (RAG), clustering**

I. Introduction

Large Language Models (LLMs) have demonstrated transformative capabilities across a wide range of natural language processing (NLP) tasks, including question answering, summarization, translation, and code generation [1]. With the emergence of foundation models trained on massive corpora, LLMs now exhibit impressive few-shot and zero-shot reasoning abilities that extend far beyond traditional supervised pipelines. These advancements have catalyzed the adoption of LLMs in domains where structured data is scarce but textual patterns are abundant, such as medical decision-making, cybersecurity incident response, and large-scale IT system monitoring [2].

In particular, system log analysis represents a high-impact application area for LLMs [3]. Logs generated by cloud platforms, networked systems, and infrastructure software encode a wealth of operational insights in semi-structured text. While other artificial intelligence approaches can handle these relevant tasks [4]–[6], LLMs are particularly compelling as general-purpose log analyzers because they can adapt to new log templates, detect abnormal patterns, and provide explanations in natural language [3].

In this context, Retrieval-Augmented Generation (RAG) has become an increasingly popular paradigm. RAG enables LLMs to access relevant external information during inference, thereby improving their ability to make accurate predictions in dynamic or specialized domains [7]. For log anomaly detection, this typically involves retrieving semantically similar historical logs and conditioning the output of the model on these examples. Such retrieval-augmented inference is especially appealing in settings with limited training data, high variance in log structure, or fast-evolving operational patterns. Despite its potential, the effectiveness of RAG-based log anomaly detection remains highly sensitive to the composition of the retrieved examples. Unlike traditional retrieval tasks that prioritize relevance or lexical similarity, LLM-based classification depends heavily on the structure, diversity, and representativeness of the examples used for prompting [1], [3].

However, poorly chosen retrievals may lead to hallucinated reasoning or misclassifications, particularly under constrained retrieval budgets. This raises a fundamental question: *What makes a good retrieval set for LLM-based log reasoning?* In this work, we investigate this question through the lens of semantic sampling strategies. Specifically, we propose a set of clustering-informed retrieval methods based on K-Means clustering in the embedding space of system logs. These methods include centroid-based, edge-based, and hybrid sampling, which aim to capture both the dense cores and peripheral outliers of the log distribution. The goal is to provide the LLM with a retrieval set that balances relevance with structural diversity and enhance its generalization in log anomaly detection.

To evaluate the effectiveness of these strategies, we conduct extensive experiments on real-world OpenStack logs using two open-source LLMs hosted locally via *Ollama*: *Granite3.2:8B* [8] and *Mistral-Nemo:12B* [9]. Additionally, we compare our clustering-informed approaches with standard baselines such as random sampling and full-pool inclusion across multiple evaluation setups. Our results show that the proposed clustering-informed strategies consistently improve F1 score, recall, and overall accuracy. The hybrid strategy, in particular, offers the best balance between precision and robustness, demonstrating that retrieval structure plays a pivotal role in RAG-based classification. In summary, our contributions are threefold: (i) we formalize the impact of semantic sampling on retrieval-augmented log anomaly detection; (ii)

we introduce novel clustering-informed strategies for selecting diverse and representative examples; and (iii) we empirically validate their effectiveness on realistic logs using two distinct LLMs. Our findings suggest new design principles for retrieval construction in LLM-based reasoning systems and point to the broader value of structure-aware prompting in operational AI.

The rest of this paper is organized as follows. Section II reviews related work. Section III formulates the problem. Section IV introduces our proposed methodology. Section V presents the experimental setup and results. Section VI concludes the work.

II. Related Work

A. LLMs for Log Analysis and Detection

Recent advances in LLMs have led to their adoption in a variety of system-related tasks, including log parsing, fault localization, and log anomaly detection [10]. Unlike traditional approaches based on rule matching or statistical thresholds, LLMs offer the potential for semantic understanding and reasoning over unstructured log data. This is particularly useful in modern IT environments where log formats are heterogeneous, logs are noisy, and anomalies may appear in subtle contextual forms [11]. While most existing works in log anomaly detection rely on conventional machine learning or deep learning classifiers [12], recent studies have started exploring the use of LLMs to directly classify or summarize logs [3]. However, purely prompt-based LLM classification suffers from performance instability, especially when lacking strong contextual grounding. To address this, RAG has been proposed as a way to supplement model predictions with relevant historical examples [13]. RAG enables models to condition their output on retrieved context, providing a flexible inference-time mechanism for adapting to new log patterns without retraining. However, prior work typically adopts off-the-shelf retrieval via embedding similarity [13], without investigating the structure or representativeness of the retrieved set. Our work fills this gap by systematically studying how the selection of retrieved examples affects anomaly classification performance.

B. Semantic Retrieval and Sampling for LLM Prompting

The quality of retrieved or selected examples plays a critical role in the effectiveness of LLM reasoning. In few-shot prompting and RAG settings, prior work has shown that the choice of examples can significantly impact performance, particularly for tasks involving classification, reasoning, or multi-turn interaction [13]. Most retrieval methods are based on nearest-neighbor search in the embedding space, assuming that semantic similarity is sufficient to guide the model. However, this often leads to highly redundant or narrow retrieval sets that may not generalize well. To address this, recent studies have explored alternative strategies for prompt example selection, including diversity-aware sampling, embedding clustering, or optimization-based prompt construction [14]. In particular, clustering methods such as K-Means have been used in other NLP tasks to identify representative or edge-case inputs, though their use in retrieval-based LLM reasoning remains limited. Our approach builds on this line of work by applying clustering-informed sampling to RAG-based log classification. Unlike prior studies that focus on relevance alone, we consider both the geometric structure and coverage of the semantic space.

III. Problem Formulation

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ to be a well labeled log dataset, where x_i is a structured log entry and $y_i \in \{\text{NORMAL}, \text{ABNORMAL}\}$ is the corresponding ground-truth label. The objective is to classify an unseen query log x_q using a large language model M_θ without parameter updates. Classification is performed by conditioning the LLM on a set of k retrieved examples drawn from a smaller index $\mathcal{D}_{\text{index}} \subset \mathcal{D}$, selected in advance.

This setting differs from conventional supervised learning in two key ways: (1) model parameters θ remain frozen, and (2) contextual supervision is delivered at inference time via prompt examples. As a result, performance is highly sensitive to the semantic structure of the retrieved context, especially under tight retrieval budgets (i.e., small k values). This motivates a structure-aware design of the retrieval index.

IV. Methodology

We propose a clustering-informed, retrieval-augmented classification framework for zero-shot log anomaly detection. The method embeds labeled logs into a dense semantic space, clusters them per class, and samples representative examples for constructing a compact but semantically rich retrieval index. At inference time, top-k examples are retrieved to form a prompt for the LLM. The proposed framework is illustrated in Figure 1.

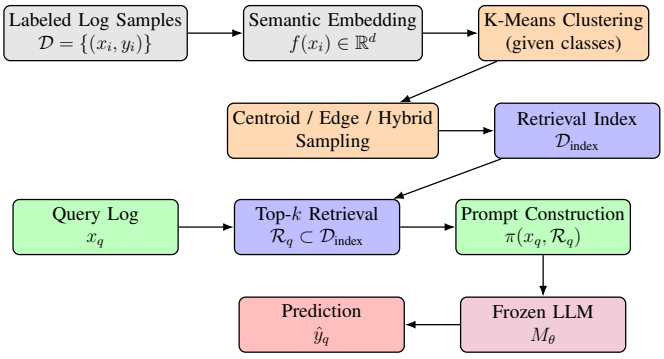

Figure 1. Pipeline overview of our clustering-informed retrieval-augmented classification framework. Log entries are embedded and clustered by class to construct a semantically structured retrieval index. At inference time, each query log retrieves top-k similar examples to form a prompt, which is fed into a frozen LLM for zero-shot classification.

Specifically, the system operates in three modular stages: (1) semantic embedding and class-wise clustering of a labeled log corpus to construct the retrieval index $\mathcal{D}_{\text{index}}$; (2) top-$k$ similarity-based retrieval from the index; and (3) prompt-based inference using a frozen LLM for classification.

A. Semantic Embedding and Similarity

To facilitate semantically meaningful retrieval, all log entries in $\mathcal{D}$ are first embedded into a shared latent space using a sentence-level encoder, e.g., *BAAI/bge-small-en-v1.5* [15]. This encoder maps each structured log x to a dense vector representation $f(x) \in \mathbb{R}^d$, where d is the dimensionality of the embedding space. The embedding function $f : \mathcal{X} \to \mathbb{R}^d$ is assumed to preserve high-level semantic similarity between log messages. Retrieval is based on cosine similarity, defined as:

$$\text{sim}(x, x') = \frac{\langle f(x), f(x') \rangle}{\|f(x)\| \cdot \|f(x')\|}, \tag{1}$$

which quantifies the angular proximity between two embedded vectors. A reduced subset $\mathcal{D}_{\text{index}} = \{x'_i\}_{i=1}^m$ is maintained as the retrieval pool. At inference time, for a given query log x_q, the top-k most similar entries are selected by maximizing the aggregated similarity.

While semantic embedding provides a meaningful similarity space for log retrieval, the quality of the retrieved examples is fundamentally limited by the composition of the retrieval pool. In particular, under small top-k budgets, uniformly sampling from the dataset can lead to redundant or uninformative context. To address this, we introduce a clustering-based strategy to construct a structured and diverse retrieval index, thereby improving the effectiveness of downstream prompting.

B. Clustering-Informed Index Construction

In our retrieval-augmented inference framework, the effectiveness of contextual prompting relies heavily on the quality and diversity of the retrieval index. We hypothesize that structuring $\mathcal{D}_{\text{index}}$ through clustering significantly improves the semantic coverage and representativeness of retrieved examples, especially under tight top-k constraints. To this end, we apply K-Means clustering in the embedding space to guide the construction of $\mathcal{D}_{\text{index}}$. We first embed each labeled log using a sentence-level encoder $f(x_i) \in \mathbb{R}^d$, and then cluster the NORMAL and ABNORMAL subsets of $\mathcal{D}$ separately. Let M denote the number of clusters per class, yielding two disjoint cluster sets. Each cluster C_j is associated with a centroid μ_j, computed as the mean of the embeddings in that cluster.

From these clusters, we sample representative examples using one of three strategies: centroid sampling, edge sampling, or hybrid sampling. In centroid sampling, we select the log closest to the cluster center:

$$x_j^{\text{centroid}} = \arg\min_{x \in C_j} \|f(x) - \mu_j\|_2. \tag{2}$$

This method captures semantically dense regions and selects prototypical logs that represent high-frequency behavior. In edge sampling, we instead select the most peripheral log:

$$x_j^{\text{edge}} = \arg\max_{x \in C_j} \|f(x) - \mu_j\|_2. \tag{3}$$

These examples reflect outliers or rare patterns, particularly useful for enriching the diversity of ABNORMAL class examples. Hybrid sampling combines both strategies by selecting both the centroid and edge from each cluster, denoted as $\mathcal{D}_{\text{index}}^{\text{hybrid}} = \bigcup_{j=1}^{M} \{x_j^{\text{centroid}}, x_j^{\text{edge}}\}$. The resulting index contains $2M$ examples per class, balancing central and boundary behaviors to increase generalization. The hyperparameter M controls the granularity of this semantic partitioning: larger values offer finer diversity at the cost of index size. In practice, we ensure that $\mathcal{D}_{\text{index}}$ remains smaller than the size of the full dataset $\mathcal{D}$, enabling efficient top-k retrieval at inference time. This structured index serves as the retrieval module in our framework and directly support downstream prompt construction and model inference.

C. Prompt Construction and Inference

Our complete system follows a RAG architecture adapted to log classification. The retrieval module consists of the semantically structured index $\mathcal{D}_{\text{index}}$ constructed in the previous stage via clustering-informed sampling. This module allows fast, label-aware, and diverse selection of relevant examples for each query. The generation module is realized via prompt-based inference using a frozen LLM, where the retrieved examples act as analogical supervision.

At inference time, we first embed the query log x_q using the same encoder f, and perform a top-k similarity search against $\mathcal{D}_{\text{index}}$ using cosine similarity. This yields a set of k nearest neighbors $\mathcal{R}_q = \{x_1, \ldots, x_k\}$, which include both normal and abnormal examples. These examples are then formatted into a few-shot prompt using a fixed template where each example is shown as a labeled log. An example of the prompt template is given as follows, where detailed log contents are partially truncated for readability, denoted by <...>.

An Example of Prompt Template used during Inference

```
System: You are an expert in analyzing
OpenStack logs.

User: Here are some log examples with known
classifications (NORMAL or ABNORMAL):

[NORMAL]: nova-compute.log.1.2017-05-16_
13:55:31 2017-05-16 00:02:21.664 2931 INFO
nova.compute.manager <...>
[ABNORMAL]: nova-compute.log.2017-05-14_
21:27:09 2017-05-14 20:50:06.614 2931 INFO
nova.compute.manager <...>
...

Now analyze the following new log entry
referring to the examples above:

nova-compute.log.1.2017-05-16_ 13:55:31
2017-05-16 00:45:14.134 2931 INFO
nova.compute.manager <...>

Classify it using only one word: either
NORMAL or ABNORMAL.
```

This prompt $\pi(x_q, \mathcal{R}_q)$ is submitted to the frozen language model M_θ, which produces a textual prediction. The output is then parsed to obtain the final label $\hat{y}_q = \text{parse}(M_\theta(\pi(x_q, \mathcal{R}_q)))$. We fix the prompt format throughout all experiments and shuffle the example order within each class

to reduce positional bias. Crucially, this RAG-style design enables flexible zero-shot classification: the model performs inference using retrieved demonstrations without any parameter updates. All modules operate at inference time and remain decoupled, which supports rapid deployment across log formats, domains, or backbone models. The detailed procedures of the proposed framework are summarized in Algorithm 1.

Algorithm 1: The proposed clustering-informed retrieval-augmented log classification.

Input : Labeled logs $\mathcal{D} = \{(x_i, y_i)\}$; encoder $f(\cdot)$; retrieval size k; target index size T; sampling strategy $\mathcal{S} \in \{\text{centroid}, \text{edge}, \text{hybrid}\}$; query log x_q; LLM $\mathcal{M}_\theta$

Output: Predicted label $\hat{y}_q$

Index Construction:
Split $\mathcal{D}$ by class into $\{\mathcal{D}_c\}$;
foreach $\mathcal{D}_c$ **do**
  $\mathcal{E}_c \leftarrow \{f(x) \mid x \in \mathcal{D}_c\}$ //Compute embedding set;
  $M \leftarrow T$ if $\mathcal{S} \neq$ hybrid else $T/2$;
  Cluster $\mathcal{E}_c$ into M clusters via K-Means;
  foreach *cluster* C_j *with centroid* μ_j **do**
    if $\mathcal{S} \neq$ *edge* **then**
      add $\arg\min_{x \in C_j} \|f(x) - \mu_j\|$ to index
    if $\mathcal{S} \neq$ *centroid* **then**
      add $\arg\max_{x \in C_j} \|f(x) - \mu_j\|$ to index

Inference:
$z_q \leftarrow f(x_q)$;
Retrieve top-k from index by cosine similarity to z_q;
Form prompt $\pi(x_q, \mathcal{R}_q)$ with retrieved examples;
$\hat{y}_q \leftarrow \text{parse}(\mathcal{M}_\theta(\pi(x_q, \mathcal{R}_q)))$;
return $\hat{y}_q$

V. Experiments

A. Dataset

Our experiments are based on the OpenStack log dataset from the LogHub benchmark [16], a large-scale collection designed for AI-driven system log analytics. OpenStack serves as a widely adopted open-source cloud platform that manages computing, networking, and storage resources. The logs were collected from CloudLab, where both normal operations and abnormal behaviors were generated through controlled failure injection. In each experimental run, we construct a balanced test set by randomly sampling 50 NORMAL and 50 ABNORMAL logs. The remaining portion of the dataset is used to build the retrieval index.

B. Experimental Setup

1) Embedding and Similarity Indexing: To emulate realistic log monitoring conditions, we adopt a retrieval-augmented framework comprising two key components: a semantically structured retrieval index and a frozen large language model for inference. All log entries are first encoded into a shared latent space using a sentence-level embedding model, specifically *BAAI/bge-small-en-v1.5*, which is well-suited for semantic similarity tasks. The resulting embeddings are stored in a FAISS index to enable efficient top-k retrieval based on cosine similarity.

2) Retrieval Pool and Index Configurations: Unless otherwise noted, the retrieval index is constructed from a fixed candidate pool consisting of 8,000 NORMAL and 2,000 ABNORMAL logs aside from the 100 logs in the test set sampled initially, for each run. This configuration, which includes all available labeled samples aside from the test set, is referred to as the full index. It serves as the upper-bound reference setting and supports retrieval from the most semantically complete and diverse log base. We also evaluate reduced index settings, where the candidate pool is downsampled to simulate constrained retrieval scenarios. Specifically, we consider reduced pools with sizes of 2000, 1000, and 500 logs, as opposed to the reduction ratio of 80%, 90%, and 95%, each sampled using different strategies to construct the reduced retrieval index.

Specifically, four index construction strategies are examined. In the random sampling baseline, logs are selected uniformly from the candidate pool without any semantic filtering. The proposed clustering-informed approaches instead apply K-Means clustering separately on the normal and abnormal subsets in the embedding space. Centroid sampling selects the log closest to each cluster center, capturing common or prototypical patterns. Edge sampling selects the log furthest from the center, promoting inclusion of peripheral or rare examples. The hybrid strategy includes both types of samples from each cluster, aiming to balance representativeness and diversity. The number of clusters per class is selected based on the desired index size.

Table I
Summary of experimental configurations.

Component	Setting
Test Set Size	100 logs (50 each class) per run
Retrieval Pool Sizes	500, 1000, 2000, 10000 (full index)
Sampling Strategies	Centroid, Edge, Hybrid, Random
Clustering Method	K-Means
Embedding Model	*BAAI/bge-small-en-v1.5*
Similarity Metric	Cosine similarity (via FAISS)
Top-k Values	k = {3, 5}
LLMs Evaluated	*Granite3.2:8B* *Mistral-Nemo:12B*
Local LLM Backend	*Ollama v0.7.1*

3) Top-k Retrieval and LLM Prompting: At inference time, each test query log is embedded using the same encoder and compared against the constructed index to retrieve the top-k most similar logs, with k set to either 3 or 5. These retrieved examples are formatted as few-shot demonstrations in a fixed natural language prompt template and submitted to a frozen LLM for classification. We evaluate two competitive open-source language models in this setup: *Granite3.2:8B* and *Mistral-Nemo:12B*. The prompt remains consistent across all

Table II
SUMMARY OF THE DETECTION PERFORMANCE UNDER FULL INDEX.

Model / Top-k	Accuracy			F1 Score			Precision			Recall		
	Min.	Avg.	Max.	Min.	Avg.	Max.	Min.	Avg.	Max.	Min.	Avg.	Max.
Granite3.2:8B (k=3)	0.950	0.970	1.000	0.947	0.969	1.000	1.000	1.000	1.000	0.900	0.940	1.000
Granite3.2:8B (k=5)	0.980	0.989	1.000	0.980	0.989	1.000	1.000	1.000	1.000	0.960	0.978	1.000
Mistral-nemo:12B (k=3)	0.953	0.969	0.990	0.951	0.968	0.990	1.000	1.000	1.000	0.907	0.944	0.980
Mistral-nemo:12B (k=5)	0.967	0.981	1.000	0.966	0.981	1.000	1.000	1.000	1.000	0.933	0.962	1.000

runs, and the order of examples within the prompt is randomly shuffled to mitigate positional bias. To ensure robust evaluation and account for variance due to random sampling, each experimental configuration, including model, top-k, sampling strategy, and index size, is repeated three times with different random seeds. All reported results are averaged across these runs, with min–max ranges also provided.

To provide a complete view of the experimental setting, we summarize the key variables and configurations used throughout the study in Table I. This includes dataset partitioning, retrieval parameters, sampling strategies, and LLM backbones. Such structured documentation clarifies the design space of our evaluation and supports reproducibility.

4) Evaluation Metrics: We evaluate model performance using four standard metrics: precision, recall, F1 score, and accuracy. Precision measures the proportion of predicted ABNORMAL logs that are truly abnormal, while recall reflects the proportion of actual ABNORMAL logs that are correctly identified. The F1 score is the harmonic mean of precision and recall, which provides a balanced view of detection quality. Accuracy represents the overall percentage of correctly classified logs.

C. Results

1) Detection Performance with Full Index: Table II summarizes the detection performance when using the complete reference index without any downsampling. All models exhibit near-perfect precision across trials, with most performance differences attributed to variations in recall. The *granite3.2:8b* model with k=5 achieves the highest overall performance, reaching an average accuracy and F1 score of 0.989. The *mistral-nemo:12b* model also performs competitively, especially with k=5, where it achieves 0.981 in both average accuracy and F1 score. These results suggest that increasing top-k value generally improves robustness and recall when the full reference set is available.

2) Impact of Index Reduction: Figure 2 illustrates how model performance degrades when the reference index is down-sampled with reduction ratios of 80%, 90%, and 95% of its original size.

Specifically, the horizontal dashed lines represent baseline accuracy under the full index. As the reduction ratio increases, all models experience performance drops, with the degradation more significant under more aggressive downsampling. Notably, *granite3.2:8b* demonstrates greater resilience, retaining

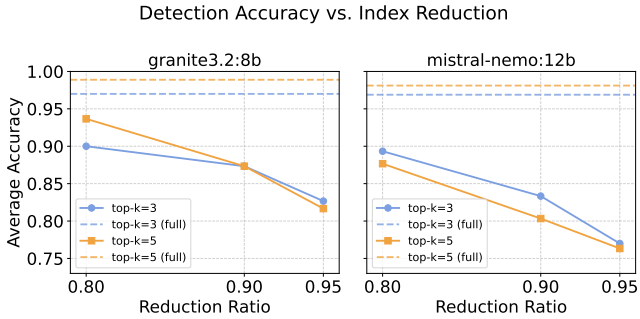

Figure 2. Impact of retrieval index reduction on detection accuracy for different LLMs.

an accuracy of around 0.94 at a 80% reduction ratio, compared to 0.88 for *mistral-nemo:12b* at the same setting. These results highlight the trade-off between index size and detection quality, and demonstrate the relative robustness of different LLM configurations.

3) Effectiveness of Clustering-Informed Index: Figure 3 further evaluates the effect of clustering-informed strategies on detection performance under limited index sizes. Specifically, we compare three proposed sampling methods, including centroid, edge, and hybrid sampling, against the random sampling baseline at varying index sizes: 500, 1000, and 2000.

The left panel reports the average accuracy with error bars computed from observed min-max spans across runs, while the right panel presents the corresponding F1 scores. As index size increases, all methods show improved performance, but the hybrid strategy consistently outperforms random sampling across all configurations. Notably, when the index size is reduced to 500, the hybrid sampling still maintains competitive accuracy and F1 score compared to random sampling at size 1000, suggesting better representation quality. Edge sampling also performs well, especially at larger index sizes, while centroid sampling is more stable across both top-k settings. These findings validate the effectiveness of clustering-informed strategies in preserving semantic diversity and discriminative power, especially under constrained index budgets.

D. Discussion

Our experimental results substantiate the central hypothesis of this work such that the retrieval structure plays a decisive role in LLM-based log anomaly detection. The proposed clustering-informed sampling ensures that the constructed

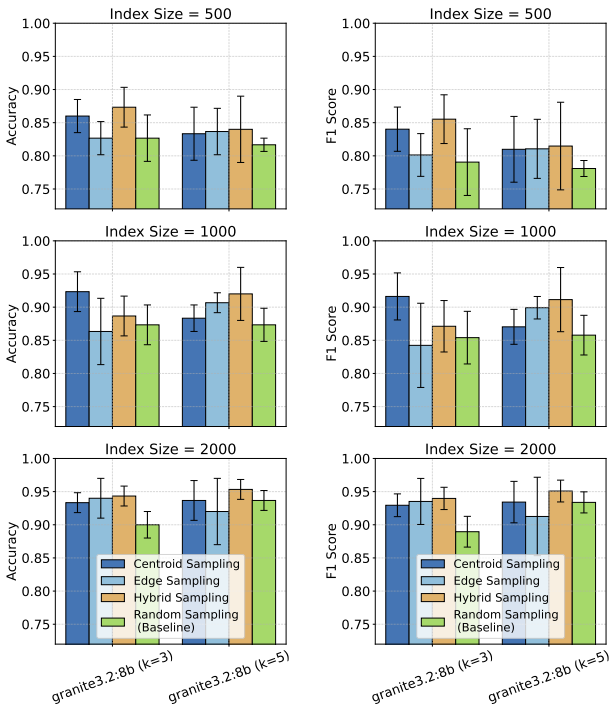

Figure 3. Comparison of detection accuracy (left) and F1 score (right) under different index sizes and sampling strategies. Each bar corresponds to a specific strategy (Centroid, Edge, Hybrid, Random), with results shown for two models (*granite3.2:8b* with k=3 and k=5). Error bars represent half the span between the minimum and maximum scores across test runs.

retrieval index captures both dense and sparse regions of semantic distribution. This enables the language model to generalize better under tight top-k budgets, where the quality of each retrieved example becomes especially critical.

Among the proposed strategies, the hybrid approach consistently outperforms all others across index sizes of 500, 1000, and 2000, particularly in the low-resource regime. This confirms the intuition that a combination of core (centroid) and boundary (edge) examples offers a more balanced semantic context for LLM reasoning. For instance, at index size 500, hybrid sampling achieves comparable accuracy and F1 score to random sampling with 1000 logs, and it highlights its superior efficiency in context construction. Interestingly, we observe that as the index size decreases, the performance gap between clustering-informed and random sampling widens. This suggests that structured sampling is not merely a refinement, but a critical factor in maintaining performance under constrained retrieval budgets. Despite these gains, the clustering step introduces non-negligible computational overhead, particularly during index construction. K-Means clustering must be applied separately to each class over thousands of embeddings, which may limit scalability in real-time or streaming settings.

VI. Conclusion

In this work, we investigated the impact of retrieval sampling strategies on LLM-based log anomaly detection within the RAG framework. While prior work often treats the retrieval process as fixed or purely relevance-driven, we showed that the structure of the retrieved context plays a critical role in guiding LLM reasoning. To this end, we introduced a family of clustering-informed retrieval strategies based on K-Means, including centroid, edge, and hybrid sampling. These methods aim to provide the model with semantically representative and diverse examples, improving its ability to distinguish between normal and anomalous patterns. Through extensive experiments on OpenStack logs and two competitive open-source LLMs, we demonstrated that clustering-informed sampling consistently improves classification performance, especially in low top-k value cases where retrieval quality is most critical. In future, we will explore adaptive clustering, uncertainty-aware retrieval, or joint optimization of retrieval and generation for more robust reasoning in log-based systems.

References

[1] A. Mohammed and R. Kora, "A comprehensive overview and analysis of large language models: Trends and challenges," *IEEE Access*, pp. 1–1, 2025.

[2] J. Luo, W. Zhang, Y. Yuan, Y. Zhao, J. Yang, Y. Gu, B. Wu, B. Chen, Z. Qiao, Q. Long *et al.*, "Large language model agent: A survey on methodology, applications and challenges," *arXiv preprint arXiv:2503.21460*, 2025.

[3] Y. Liu, S. Tao, W. Meng, F. Yao, X. Zhao, and H. Yang, "Logprompt: Prompt engineering towards zero-shot and interpretable log analysis," in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, 2024, pp. 364–365.

[4] J. Zhang, F. Li, and F. Ye, "Sustaining the high performance of ai-based network traffic classification models," *IEEE/ACM Transactions on Networking*, vol. 31, no. 2, pp. 816–827, 2023.

[5] M. Ali and J. Zhang, "Exploring the effectiveness of synthetic data in network intrusion detection through xai," in *2024 Cyber Awareness and Research Symposium (CARS)*, 2024, pp. 1–5.

[6] J. Zhang, F. Li, and F. Ye, "An ensemble-based network intrusion detection scheme with bayesian deep learning," in *ICC 2020 - 2020 IEEE International Conference on Communications (ICC)*, 2020, pp. 1–6.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in neural information processing systems*, vol. 33, pp. 9459–9474, 2020.

[8] I. Research, "Granite-3.2:8b," https://ollama.com/library/granite3.2, 2025, large language model fine-tuned for reasoning tasks. Available via Ollama.

[9] M. AI and NVIDIA, "Mistral nemo 12b," https://ollama.com/library/mistral-nemo, 2024, state-of-the-art 12B language model with 128k context length. Available via Ollama.

[10] X. Huang, T. Zhang, and W. Zhao, "Logrules: Enhancing log analysis capability of large language models through rules," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 452–470.

[11] A. Elhafsi, R. Sinha, C. Agia, E. Schmerling, I. A. Nesnas, and M. Pavone, "Semantic anomaly detection with large language models," *Autonomous Robots*, vol. 47, no. 8, pp. 1035–1055, 2023.

[12] H. Studiawan, F. Sohel, and C. Payne, "Anomaly detection in operating system logs with deep learning-based sentiment analysis," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 5, pp. 2136–2148, 2020.

[13] J. Pan, W. S. Liang, and Y. Yidi, "Raglog: Log anomaly detection using retrieval augmented generation," in *2024 IEEE World Forum on Public Safety Technology (WFPST)*. IEEE, 2024, pp. 169–174.

[14] A. Sabbatella, A. Ponti, I. Giordani, A. Candelieri, and F. Archetti, "Prompt optimization in large language models," *Mathematics*, vol. 12, no. 6, p. 929, 2024.

[15] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-pack: Packaged resources to advance general chinese embedding," 2023.

[16] C. by LOGPAI, "Loghub: A large collection of system log datasets for ai-driven log analytics," Jul. 2023. [Online]. Available: https://doi.org/10.5281/zenodo.8196385